# HDOC Day-52 Activity

**Previous Greater Element Using Pointers and Nested Loops**

| | |
|---|---|
| **Student Name** | Dhruv Pandey |
| **SAP ID** | 590037660 |
| **Batch** | 15 |
| **Course / Activity** | HDOC Day-52 |

## Aim

To write a C program that finds the nearest previous greater element for every element of a variable-length integer array by passing the array to **previous_greater()** using its address and accessing it through pointers.

## Problem Requirements

- Read the size and elements of a variable-length integer array.
- Pass the array to **previous_greater()** using its address.
- Search only the elements to the **left** of the current element.
- Select the **nearest** element that is greater than the current element.
- Print **-1** when no greater element exists on the left.
- Display answers in array order, separated by commas.
- Use a **nested-loop** approach.

## Algorithm

1. Read the array size **n**.
2. Read **n** integer elements into **arr[]**.
3. Pass the array address and size to **previous_greater(arr, n)**.
4. For every index **i**, initialize **previous = -1**.
5. Start an inner loop from **j = i - 1** and move left using **j--**.
6. If ***(arr + j) > *(arr + i)**, store that value and immediately break. This ensures the nearest greater element is selected.
7. Print the stored value. Repeat for all elements.

## C Program

```c
#include <stdio.h>

void previous_greater(int *arr, int n)
{
    for (int i = 0; i < n; i++)
    {
        int previous = -1;

        for (int j = i - 1; j >= 0; j--)
        {
            if (*(arr + j) > *(arr + i))
            {
                previous = *(arr + j);
                break;
            }
        }

        printf("%d", previous);

        if (i < n - 1)
        {
            printf(", ");
        }
    }
}

int main()
{
    int n;

    printf("Enter the size of array: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter the elements: ");

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Previous greater elements: ");
    previous_greater(arr, n);

    return 0;
}
```

## Pointer Usage

**previous_greater(arr, n)** passes the address of the first array element to the function. The parameter **int *arr** receives that address. The expression ***(arr + i)** accesses the same value as **arr[i]**.

## Why the Inner Loop Moves Backward

For each current element, the search starts at the immediately previous position **i - 1** and moves toward index 0. As soon as a greater value is found, **break** stops the search. Therefore, the selected value is the nearest greater element on the left.

## Test Cases / Sample Executions

| Case | Input Array | Expected Output |
|---|---|---|
| Example 1 | 4 5 2 25 | -1, -1, 5, -1 |
| Example 2 | 13 7 6 12 | -1, 13, 7, 13 |
| Example 3 | 1 3 2 4 | -1, -1, 3, -1 |
| Example 4 | 1 2 3 4 5 | -1, -1, -1, -1, -1 |

## Dry Run - Example 2

| Current | Search to the left | Nearest greater | Output |
|---|---|---|---|
| 13 | No element on left | None | -1 |
| 7 | 13 | 13 > 7 | 13 |
| 6 | 7 | 7 > 6 | 7 |
| 12 | 6, 7, 13 | 13 > 12 | 13 |

## Complexity

**Time Complexity:** $O(n^2)$ in the worst case because nested loops are used.

**Space Complexity:** O(1) extra space because only a few variables are used apart from the input array.

## Result

The program successfully finds and displays the nearest previous greater element for every array element using pointers and nested loops. If no greater element exists to the left, it displays -1.

## Execution Example

```
Enter the size of array: 4
Enter the elements: 13 7 6 12
Previous greater elements: -1, 13, 7, 13
```